

GraphBench Challenge

Réfutation automatique de conjectures en théorie des graphes

Étudiants :

Martim MOREIRA ALVES

Noé ARNAUD

Formation :

Master 1 MIAGE — Université Paris Dauphine

Date :

12 mai 2026

Dépôt GitHub :

https://github.com/AlvesMartim/OC_Graph

Conjectures réfutées	Score final (secondes)
----------------------	------------------------

99 / 100	221,51 s
----------	----------

Table des matières

1	Introduction et objectif	2
2	Heuristique simple	2
2.1	Vue d'ensemble	2
2.2	Phase 1 : passe atlas	3
2.3	Phase 2 : population initiale et seeds structurées	3
2.4	Phase 3 : recherche locale avec diversification	3
2.5	Calcul des invariants	5
2.6	Réparation des graphes	5
3	Architecture FunSearch	5
3.1	Principe général	5
3.2	Boucle d'évolution	6
3.3	Intégration avec l'heuristique simple	6
4	Résultats expérimentaux	6
4.1	Performance globale	6
4.2	La conjecture non réfutée	6
4.3	Distribution des temps de réfutation	7
4.4	Contribution de FunSearch	7
5	Discussion scientifique	7
5.1	Quelles conjectures sont faciles à réfuter ?	7
5.2	Quels invariants sont difficiles à manipuler ?	8
5.3	Quelles mutations sont efficaces ?	8
5.4	FunSearch améliore-t-il réellement l'heuristique ?	8
5.5	L'IA a-t-elle produit des idées utiles ?	9

1 Introduction et objectif

L’objectif de ce projet est simple : on nous donne 100 conjectures sur les graphes, et on doit en réfuter le plus possible automatiquement. Une conjecture prend la forme d’une inégalité entre deux mesures d’un graphe (appelées *invariants*) :

$$Y(G) \leq f(X(G)) \quad \text{ou} \quad Y(G) \geq f(X(G))$$

où X et Y sont par exemple le degré maximum, le nombre de triangles, ou la proximité, et f est un polynôme donné. Réfuter une conjecture, c’est trouver un graphe G pour lequel l’inégalité est fausse.

Pour mesurer à quel point un graphe “viole” une conjecture, on calcule :

$$\text{violation}(G) = \begin{cases} Y(G) - f(X(G)) & \text{si signe} \leq \\ f(X(G)) - Y(G) & \text{si signe} \geq \end{cases}$$

Dès que ce score est positif, on a trouvé un contre-exemple. Le score final du projet est la **somme des temps de réfutation**, avec une pénalité de 120 s pour chaque conjecture qu’on n’arrive pas à réfuter.

Nous avons développé deux approches. La **Partie 1** est une heuristique de recherche locale : on part de graphes connus ou aléatoires, et on les modifie progressivement pour augmenter le score de violation. La **Partie 2** utilise **FunSearch** : on demande à un LLM de générer des fonctions de guidage plus intelligentes, qu’on teste et améliore par itérations.

2 Heuristique simple

2.1 Vue d’ensemble

Pour chaque conjecture, l’heuristique suit trois étapes dans l’ordre (Figure 1). L’idée générale est de d’abord tester rapidement des graphes “classiques”, puis si ça ne suffit pas, de lancer une recherche plus longue en modifiant des graphes au fil du temps.

Phase 1	Phase 2	Phase 3
Passe atlas (≤ 5 s)	Population initiale	Boucle locale (60 s)
Graphes ≤ 7 nœuds	Seeds structurées	Tournoi $k = 3$
Arbres 8–12 nœuds	Graphes aléatoires	Mutation ciblée 70%
Familles paramétriques	(10 graphes)	Liste taboue + kick

FIGURE 1 – Pipeline de l’heuristique simple.

2.2 Phase 1 : passe atlas

Avant de faire quoi que ce soit de complexe, on teste une grande liste de graphes connus pour voir si l'un d'eux est déjà un contre-exemple :

- **Atlas NetworkX** : tous les graphes connexes à au plus 7 sommets (1 252 graphes), testés en quelques millisecondes.
- **Arbres** de 8 à 12 sommets, si la conjecture porte sur les arbres.
- **Familles classiques** pour n de 8 à 25 : chemins, cycles, étoiles, graphes complets, roues, chenilles, doubles-étoiles, bipartis, K_n + bras.

Cette étape résout à elle seule **72 des 99 conjectures réfutées**. Beaucoup de conjectures ont des contre-exemples très simples, comme K_2 (le graphe à 2 sommets reliés par une arête), trouvé en moins de 0,1 s.

2.3 Phase 2 : population initiale et seeds structurées

Si l'atlas n'a rien trouvé, on construit une population de départ de 10 graphes pour amorcer la recherche. Une partie de ces graphes est choisie *intelligemment* selon les invariants de la conjecture :

- Si la conjecture parle de **remoteness** : on part de graphes K_n + bras (une clique avec un chemin accroché), qui ont une remoteness très basse.
- Si la conjecture parle de **proximity** : on part de doubles-étoiles très déséquilibrées ou de “brooms” (chemin + étoile à un bout), qui étirent le graphe et font baisser la proximity.

Le reste est généré aléatoirement en respectant la classe de graphes demandée (**connected**, **tree**, ou **claw_free**).

2.4 Phase 3 : recherche locale avec diversification

La boucle principale modifie les graphes de la population un par un, en gardant ceux qui progressent vers une violation positive. Trois mécanismes évitent de rester bloqué au même endroit :

Algorithme 1 : Recherche locale avec liste taboue, kick et redémarrage

Input : Conjecture C , budget temps $T = 60$ s

Output : Contre-exemple G^* ou échec

```
1 tabu  $\leftarrow \emptyset$ ;
2 pop  $\leftarrow$  GenerateInitialGraphs( $C$ );
3 Évaluer chaque  $G \in$  pop, conserver les 10 meilleurs;
4 while temps écoulé  $< T$  do
5    $G \leftarrow$  TournoiSelection(pop,  $k = 3$ );
6   if stagnation  $> KICK$  (100 iter) then
7      $H \leftarrow$  KickMutate( $G$ , 4 mutations);
8   else
9      $H \leftarrow$  TargetedMutate( $G$ ,  $C$ );
10  end
11   $H \leftarrow$  Repair( $H$ , classe( $C$ ));
12  if graph6( $H$ )  $\in$  tabu then
13    continue;
14  end
15  Ajouter graph6( $H$ ) dans tabu;
16  Calculer violation( $H$ ,  $C$ );
17  if violation( $H$ )  $> 0$  then
18    return  $H$ ;
19  end
20  Mettre à jour pop;
21  if stagnation  $> RESTART$  (500 iter) then
22    pop  $\leftarrow$  GenerateInitialGraphs( $C$ );
23    stagnation  $\leftarrow 0$ ;
24  end
25 end
26 return échec;
```

Sélection par tournoi. On tire 3 graphes au hasard dans la population et on choisit le meilleur. Cela favorise les bons graphes sans pour autant ignorer les autres.

Mutations ciblées. 70% du temps, la modification est *guidée* par la conjecture : si on veut augmenter Y , on applique une mutation conçue pour ça. Par exemple :

- Augmenter **triangle_number** : on cherche deux voisins d'un même sommet qui ne sont pas encore reliés, et on ajoute l'arête.
- Augmenter **diameter** : on accroche un nouveau sommet à une feuille existante pour

allonger le graphe.

- Diminuer **proximity** : on ajoute un sommet à l’endroit le plus “loin” du reste du graphe, pour l’étirer encore plus.
- Augmenter **clique_number** : on connecte un sommet extérieur à tous les membres de la plus grande clique actuelle.

Les 30% restants sont des mutations aléatoires (ajout/suppression d’arête ou de sommet) pour maintenir la diversité.

Liste taboue. Chaque graphe visité est converti en une chaîne **graph6** unique et mémorisé. Si on retombe sur un graphe déjà vu, on l’ignore. Cela évite de tourner en rond.

Kick et redémarrage. Si le score ne progresse pas depuis 100 itérations, on applique 4 mutations d’un coup pour sortir du blocage (*kick*). Si ça ne suffit toujours pas après 500 itérations, on repart de zéro avec une nouvelle population.

2.5 Calcul des invariants

On calcule 25 invariants différents, mais seulement ceux dont la conjecture a besoin (calcul *lazy*). Les deux invariants de distance les plus utilisés sont définis ainsi : si T_v est la somme des distances de v à tous les autres sommets,

$$\text{proximity}(G) = \frac{n-1}{\max_v T_v}, \quad \text{remoteness}(G) = \frac{n-1}{\min_v T_v}.$$

Les invariants spectraux (valeur propre maximale, connectivité algébrique) sont calculés avec NumPy. Les invariants combinatoires comme le nombre de domination utilisent des algorithmes de NetworkX.

2.6 Réparation des graphes

Une mutation peut casser une propriété requise (par exemple déconnecter le graphe). Après chaque mutation, on répare automatiquement :

- **connected** : on relie les composantes déconnectées par des arêtes aléatoires.
- **tree** : on calcule un arbre couvrant (algorithme de Kruskal).
- **claw_free** : on supprime les arêtes qui créent des griffes $K_{1,3}$.

3 Architecture FunSearch

3.1 Principe général

FunSearch [?] est une technique proposée par Google DeepMind où un LLM génère du code, ce code est testé automatiquement, et les meilleurs résultats sont réinjectés dans le

prompt pour guider la prochaine génération. Ici, on l’applique de la façon suivante : au lieu de coder à la main comment guider la recherche, on demande au LLM de générer une *fonction de score* Python qui remplace notre score de violation dans la boucle de recherche.

3.2 Boucle d’évolution

La boucle tourne 10 fois et suit toujours le même enchaînement :

1. **Génération.** On envoie un prompt à `gemini-2.5-flash-lite` qui décrit le problème, les meilleures fonctions trouvées jusqu’ici, et les conjectures que personne n’a encore réfutées. Le LLM renvoie une fonction Python `heuristic_score(G, inv, conjecture)`.
2. **Compilation.** On essaie d’exécuter le code. S’il contient une erreur de syntaxe, on passe à l’itération suivante.
3. **Évaluation.** La fonction guide la recherche sur 25 conjectures choisies aléatoirement (60 s max par conjecture, en parallèle). On mesure le coût total (temps de réfutation + 120 s de pénalité par échec).
4. **Sélection.** On garde les 3 meilleures fonctions (coût le plus bas).
5. **Feedback.** Ces 3 fonctions sont incluses dans le prompt suivant, avec la liste des conjectures toujours non réfutées.

3.3 Intégration avec l’heuristique simple

La Partie 2 réutilise tout le code de la Partie 1 : même génération de population, mêmes mutations, même réparation des graphes. On remplace uniquement la fonction qui évalue si un graphe est “prometteur”. La vérification finale (violation > 0) reste celle de la Partie 1, ce qui garantit que les contre-exemples produits sont bien réels.

4 Résultats expérimentaux

4.1 Performance globale

La passe atlas est de loin la plus efficace : elle réfute 72 conjectures sur 99 avant même que la boucle de recherche ne commence. Les 27 restantes sont réfutées par la recherche locale, la plus longue prenant 29 s. Au total, 85% des conjectures réfutées l’ont été en moins d’une seconde.

4.2 La conjecture non réfutée

La seule conjecture qu’on n’a pas réussi à réfuter est :

Métrique	Valeur
Conjectures réfutées	99 / 100
Score total (pénalité incluse)	221,51 s
Temps moyen de réfutation	1,03 s
Temps minimal	0,032 s
Temps maximal	29,21 s
Réfutées en moins d’1 s	85 / 99
Réfutées par la passe atlas	72 / 99

TABLE 1 – Résultats de la dernière exécution (11 mai 2026).

Si G est connexe et sans griffe, $x = \text{total_domination_number}$ et $y = \text{independence_number}$, alors $y(G) \leq 1 + x(G)$.

Le problème est double : la contrainte “sans griffe” réduit beaucoup l’espace des graphes explorables, et le nombre de domination totale est coûteux à calculer. En pratique, l’inégalité semble toujours vraie pour ces graphes — il est possible que cette conjecture soit vraie et donc non réfutable.

4.3 Distribution des temps de réfutation

Les temps de réfutation sont très déséquilibrés : 85 conjectures en moins d’1 s (trouvées dans l’atlas), et 14 entre 1 et 30 s (trouvées par la recherche locale). Ce déséquilibre s’explique simplement : certaines conjectures ont des contre-exemples très petits (K_2 , K_3), d’autres demandent des graphes plus élaborés.

4.4 Contribution de FunSearch

FunSearch n’a pas changé le score final (déjà à 99/100 avec l’heuristique simple). Sur certaines conjectures difficiles, les fonctions générées par le LLM ont légèrement accéléré la recherche, mais l’impact global reste limité.

5 Discussion scientifique

5.1 Quelles conjectures sont faciles à réfuter ?

Les plus faciles sont celles qui portent sur des invariants liés à la densité : `clique_number`, `average_degree`, `triangle_number`, `maximum_degree`. Ces invariants montent très vite dans les graphes denses, donc un simple K_2 ou K_3 suffit souvent. Par exemple, la conjecture $\text{clique_number}(G) \leq -1/2 + 15/8 \cdot \bar{d}(G)$ est réfutée par K_2 en 0,06 s (violation de 0,625).

Les conjectures sur les invariants de distance (diamètre, proximité) dans la classe `connected` sont aussi plus faciles que prévu : les chemins et cycles, présents dans l’atlas,

donnent souvent des contre-exemples immédiats.

5.2 Quels invariants sont difficiles à manipuler ?

Trois catégories posent problème :

- **total_domination_number avec claw_free** : l'espace des graphes sans griffe est très restreint, et le nombre de domination totale change peu d'une mutation à l'autre. C'est la seule conjecture non réfutée.
- **second_smallest_laplace_eigenvalue** (connectivité algébrique) : cet invariant est très peu sensible aux petites modifications du graphe, ce qui rend la recherche locale lente à converger.
- **largest_distance_eigenvalue** : invariant rare, absent de NetworkX, qui demande de construire la matrice des distances et de calculer ses valeurs propres à chaque évaluation — c'est coûteux.

5.3 Quelles mutations sont efficaces ?

Les mutations les plus utiles dans notre expérience sont :

1. **Le kick (4 mutations d'un coup)** : indispensable pour sortir des blocages. Sans lui, la recherche reste coincée sur des graphes dont le score de violation est légèrement négatif et ne progresse plus.
2. **Augmenter le diamètre** (accrocher un nouveau sommet à une feuille) : une modification simple qui a un effet immédiat sur le diamètre, la proximité et la remoteness à la fois.
3. **Les seeds K_n + bras** : ces graphes (une clique avec un chemin accroché) combinent haute densité locale et longue portée, ce qui les rend utiles pour beaucoup de conjectures en même temps.

Les mutations aléatoires (30% du temps) sont aussi utiles : elles empêchent la recherche de se concentrer trop sur une seule structure.

5.4 FunSearch améliore-t-il réellement l'heuristique ?

Dans notre cas, FunSearch n'a pas amélioré le score final : l'heuristique simple était déjà à 99/100. Sur des conjectures difficiles prises séparément, les fonctions du LLM ont parfois guidé la recherche un peu plus vite, mais l'écart reste faible.

FunSearch serait probablement plus utile si l'heuristique de départ était moins efficace : c'est quand on ne sait pas comment guider la recherche que le LLM peut apporter quelque chose de nouveau.

5.5 L'IA a-t-elle produit des idées utiles ?

En partie. Parmi les idées originales générées par le LLM :

- Récompenser les graphes dont la densité dépasse un seuil adaptatif selon la conjecture, ce qui oriente vers des graphes plus denses.
- Utiliser le ratio degré max / degré min comme indicateur de l'hétérogénéité du graphe.
- Pénaliser les graphes réguliers quand la conjecture fait intervenir des invariants non linéaires.

En revanche, beaucoup de fonctions générées étaient banales (renvoyer directement le score de violation) ou carrément invalides syntaxiquement. Le LLM est utile comme source d'idées, mais pas fiable pour les appliquer correctement sans vérification.

Conclusion

On a réussi à réfuter **99 conjectures sur 100** avec un score total de **221,51 s**. L'heuristique simple fait l'essentiel du travail : la passe atlas réfute 72% des cas en quelques millisecondes, et la recherche locale avec mutations ciblées gère le reste. La seule conjecture non réfutée (sans griffe + domination totale) semble peut-être vraie.

FunSearch est une piste intéressante, mais dans ce contexte il n'a pas apporté de gain visible. Pour la suite, il serait plus prometteur de lui demander de générer directement des familles de graphes plutôt que des fonctions de score, ou de l'utiliser sur des conjectures où on n'a aucune idée de comment guider la recherche à la main.